COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CpE21S3
DATE PERFORMED:	08/04/2026	**DATE SUBMITTED:**	08/06/2026
NAME:	Wayne Kaisen C. Paraiso	**INSTRUCTOR:**	Engr. Jimlord Quejado

Hands-on Activity 5.1 Queues

1. PROCEDURE OUTPUT

ILO A:

ILOA > main.cpp > main()

```cpp
#include <iostream>
#include <queue>

void display(std::queue<char> copyQ);

int main(){
    std::queue<char> myQ;

    myQ.push('W');
    myQ.push('A');
    myQ.push('Y');
    myQ.push('N');
    myQ.push('E');
    display(myQ);

    //empty
    std::cout << "Is the queue empty?: " << myQ.empty()<< std::endl;

    //size
    std::cout << "The length of the queue is: " << myQ.size()<< std::endl;

    //front
    std::cout << "The front of the queue is: " << myQ.front()<< std::endl;

    //back
    std::cout<< "The back of the queue is: " << myQ.back()<< std::endl;

    //pop
    std::cout << "Popping the front"<< std::endl;
    myQ.pop();
    display(myQ);

    //push
    std::cout << "Pushing to the back" << std::endl;
    myQ.push('A');
    display(myQ);
    return 0;
}
```

```
39
40  void display(std::queue<char> copyQ){
41      std::queue<char> c = copyQ;
42      while(!c.empty()){
43          std::cout<<" "<<c.front();
44          c.pop();
45      }
46      std::cout<<std::endl;
47  }
```

OUTPUT:

```
 W A Y N E
Is the queue empty?: 0
The length of the queue is: 5
The front of the queue is: W
The back of the queue is: E
Popping the front
 A Y N E
Pushing to the back
 A Y N E A
```

ILOB:

main.cpp 2

ILOB > main.cpp > main()

```cpp
#include <iostream>
#include "queueLL.h"

int main(){
    qNode<char>* front = nullptr;
    qNode<char>* back = nullptr;

    enqueue('W', &front, &back);
    std::cout<< front->data <<" "<< back->data<< std::endl;

    enqueue('A', &front, &back);
    std::cout<< front->data <<" "<< back->data<< std::endl;

    enqueue('Y', &front, &back);
    std::cout<< front->data <<" "<< back->data<< std::endl;

    enqueue('N', &front, &back);
    std::cout<< front->data <<" "<< back->data<< std::endl;

    enqueue('E', &front, &back);
    std::cout<< front->data <<" "<< back->data<< std::endl;

   checkFront(front);
   checkBack(back);
   display(front);


   dequeue(&front, &back);
   display(front);
   checkFront(front);
   checkBack(back);


    return 0;
}
```

ILOB > C queueLL.h > display<T>(qNode<T>*)

```h
#ifndef QUEUELL_H
#define QUEUELL_H

template<typename T>
class qNode{
    public:
        T data;
        qNode<T>* next;
};

template <typename T>
qNode<T>* new_node(T newData){
    qNode<T>* newNode = new qNode<T>;
    newNode->data = newData;
    newNode->next = nullptr;
    return newNode;
}

template<typename T>
void enqueue(T newData, qNode<T>** frontPtr, qNode<T>** backPtr){
    qNode<T>* newNode = new_node(newData);

    if((*frontPtr) == nullptr && (*backPtr)==nullptr){
        (*frontPtr) = newNode;
        (*backPtr) = newNode;
    }

    (*backPtr)->next = newNode;
    (*backPtr) = newNode;
}

template<typename T>
void dequeue(qNode<T>** frontPtr, qNode<T>** backPtr){
    qNode<T>* deleteNode = nullptr;
    deleteNode = (*frontPtr);

    if((*frontPtr)->next == nullptr && (*backPtr)->next == nullptr){
        (*frontPtr) = nullptr;
        (*backPtr) = nullptr;
```

```
39          (*backPtr) = nullptr;
40          delete deleteNode;
41          return;
42      }
43
44      (*frontPtr) = (*frontPtr)->next;
45      deleteNode->next = nullptr;
46      delete deleteNode;
47
48  }
49
50  template<typename T>
51  void checkFront(qNode<T>* frontPtr){
52      if(frontPtr== nullptr){
53          std::cout<< "Queue is empty"<<std::endl;
54          return;
55      }
56      std::cout<<"The Front of the queue is: " << frontPtr->data<<std::endl;
57  }
58
59  template<typename T>
60  void checkBack(qNode<T>* backPtr){
61      if(backPtr== nullptr){
62          std::cout<< "Queue is empty"<<std::endl;
63          return;
64      }
65      std::cout<< "The back of the queue is: " << backPtr->data<<std::endl;
66  }
67
68  template<typename T>
69  void display(qNode<T>* frontPtr){
70      if(frontPtr == nullptr){
71          std::cout<< "Queue is empty"<<std::endl;
72          return;
73      }
74      qNode<T>* displayNode = nullptr;
```

```
75      displayNode = frontPtr;
76
77      std::cout<< "Queue List: "<< std::endl;
78
79      while(displayNode != nullptr){
80          std::cout<<displayNode->data;
81          displayNode = displayNode->next;
82      }
83      std::cout<< std::endl;
84      displayNode = nullptr;
85      delete displayNode;
86  }
87  #endif
```

```
W W
W A
W Y
W N
W E
The Front of the queue is: W
The back of the queue is: E
Queue List:
WAYNE
Queue List:
AYNE
The Front of the queue is: A
The back of the queue is: E
PS C:\msys64\ucrt64\bin>
```

Observation:
I have observed that the queue is similar to stack, the only difference of it is that queue is FIFO while stack is LIFO, I used my existing knowledged that I have learned prior to this lesson that helped me in easily adding new functions in the queue. I had a hard time with my display function because of a wrong argument when it came to the while loop, I needed it to stop when the displayNode hit a nullptr, but instead i put it as when displayNode->next = nullptr, it would stop and this would stop my program one letter short so I experimented and realized that I don't need to look if the next node is a null, i should just stop it when the node is the Null node.

Code Analysis:
The mainStl.cpp file uses the std::queue container from the STL to create a queue of characters. It pushes the letters 'W', 'A', 'Y', 'N', and 'E' into the queue one at a time, then displays the contents using a helper display() function that takes a copy of the queue so that the original is not affected when its elements are printed and removed. The program then demonstrates several built-in queue functions: empty() to check whether the queue has any elements, size() to get the number of elements currently in the queue, front() to view the element at the front, and back() to view the element at the back. After that, it uses pop() to remove the front element and push() to add a new character to the back, displaying the queue again after each operation to show how it changes.

The mainLL.cpp file, along with its header queueLL.h, implements the same idea but manually, without relying on the STL. The header defines a generic qNode structure representing a single node in a linked list, along with template functions to manage the queue: new_node() creates a new node, enqueue() adds a node to the back of the queue, and dequeue() removes a node from the front. Two additional functions, checkFront() and checkBack(), are used to display the current front and back elements of the queue, while display() traverses the entire linked list from front to back and prints out all the elements it contains. In mainLL.cpp, the same five characters are enqueued one at a time, with the front and back of the queue printed after each insertion to show how the queue grows. The program then checks and displays the full queue, removes the front element using dequeue(), and displays the queue once more to show the updated state after the removal.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Source Code (queue.h):

Supplementary > C queue.h > Printer

```
#ifndef QUEUE_H
#define QUEUE_H

#include <iostream>
#include <string>

class Job{
    public:
        int jobID, pages;
        std::string name;

        Job(){
            jobID = 0;
            name = "";
            pages = 0;
        }
        Job(int i, std::string n, int p){
            jobID = i;
            name = n;
            pages = p;
        }
};


template<typename T>
class qNode{
    public:
        T data;
        qNode<T>* next;
};

template <typename T>
qNode<T>* new_node(T newData){
    qNode<T>* newNode = new qNode<T>;
    newNode->data = newData;
    newNode->next = nullptr;
    return newNode;
```

```
38  }
39
40  template<typename T>
41  void enqueue(T newData, qNode<T>** frontPtr, qNode<T>** backPtr){
42      qNode<T>* newNode = new_node(newData);
43
44      if((*frontPtr) == nullptr && (*backPtr)==nullptr){
45          (*frontPtr) = newNode;
46          (*backPtr) = newNode;
47      }
48
49      (*backPtr)->next = newNode;
50      (*backPtr) = newNode;
51  }
52
53  template<typename T>
54  void dequeue(qNode<T>** frontPtr, qNode<T>** backPtr){
55      qNode<T>* deleteNode = nullptr;
56      deleteNode = (*frontPtr);
57
58      if((*frontPtr)->next == nullptr && (*backPtr)->next == nullptr){
59          (*frontPtr) = nullptr;
60          (*backPtr) = nullptr;
61          delete deleteNode;
62          return;
63      }
64
65      (*frontPtr) = (*frontPtr)->next;
66      deleteNode->next = nullptr;
67      delete deleteNode;
68
69  }
70
71  template <typename T>
```

```cpp
template <typename T>
bool isEmpty(qNode<T>** frontPtr){
    return (*frontPtr) == nullptr;
}

class Printer{
    private:
        qNode<Job>* front;
        qNode<Job>* back;

    public:
        Printer(){
        front = nullptr;
        back = nullptr;
    }

        void addJob(Job newJob){
        enqueue(newJob, &front, &back);
        std::cout << "Job added to the queue: " << newJob.name << std::endl;
        std::cout << "Job ID: " << newJob.jobID << std::endl;
        std::cout << "Pages: " << newJob.pages << std::endl;
    }

    void processJob(){
        if(isEmpty(&front)){
            std::cout << "No jobs in the queue to process." << std::endl;
            return;
        }
        Job jobToProcess = front->data;
        std::cout << "Processing job: " << jobToProcess.name << std::endl;
        std::cout << "Job ID: " << jobToProcess.jobID << std::endl;
        std::cout << "Pages: " << jobToProcess.pages << std::endl;
        dequeue(&front, &back);
    }

```

```
106     void processAllJobs(){
107         if(isEmpty(&front)){
108             std::cout << "No jobs in the queue to process." << std::endl;
109             return;
110         }
111         while(!isEmpty(&front)){
112             processJob();
113         }
114     }
115 };
116
117
118
119 #endif
```

Main.cpp:

Supplementary > main.cpp > main()

```cpp
#include <iostream>
#include "queue.h"

int main(){
    Printer myPrinter;

    std::cout << "Adding jobs to the queue:" << std::endl;
    myPrinter.addJob(Job(424556, "James Mahoraga", 8));
    myPrinter.addJob(Job(492387, "Alliesha Paraiso", 19));

    std::cout<<std::endl;
    std::cout <<"Processing jobs in the queue:" << std::endl;
    myPrinter.processJob();

    std::cout<<std::endl;
    std::cout <<"Adding more jobs to the queue:" << std::endl;
    myPrinter.addJob(Job(978497, "Peter Parker", 14));
    myPrinter.addJob(Job(897429, "Maynerd Parker", 18));

    std::cout<<std::endl;
    std::cout <<"Processing remaining jobs in the queue:" << std::endl;
    myPrinter.processAllJobs();

    std::cout<<std::endl;
    std::cout <<"Processing remaining jobs in the queue:" << std::endl;
    myPrinter.processAllJobs();

    return 0;
}
```

Output:

```
Adding jobs to the queue:
Job added to the queue: James Mahoraga
Job ID: 424556
Pages: 8
Job added to the queue: Alliesha Paraiso
Job ID: 492387
Pages: 19

Processing jobs in the queue:
Processing job: James Mahoraga
Job ID: 424556
Pages: 8

Adding more jobs to the queue:
Job added to the queue: Peter Parker
Job ID: 978497
Pages: 14
Job added to the queue: Maynerd Parker
Job ID: 897429
Pages: 18

Processing remaining jobs in the queue:
Processing job: Alliesha Paraiso
Job ID: 492387
Pages: 19
Processing job: Peter Parker
Job ID: 978497
Pages: 14
Processing job: Maynerd Parker
Job ID: 897429
Pages: 18
```

```
Processing remaining jobs in the queue:
No jobs in the queue to process.
```

Code Analysis:

This program simulates a simple printer job queue system using a linked list-based queue implemented with C++ templates. The queue.h file defines a Job class that stores basic information about each print job, including a job ID, the name of the person who submitted it, and the number of pages to be printed. It also defines a generic qNode structure and a set of template functions (new_node, enqueue, dequeue, and isEmpty) that allow any data type to be stored and managed in a queue using a linked list, where nodes are added at the back and removed from the front, following the First-In-First-Out (FIFO) principle.

Building on top of this generic queue, the Printer class is created specifically to manage Job objects. It keeps track of the front and back of the queue and provides three main functions: addJob(), which adds a new print job to the queue and displays its details; processJob(), which processes and removes a single job from the front of the queue, printing its information to the screen; and processAllJobs(), which repeatedly calls processJob() until the entire queue has been emptied.

In main.cpp, the program creates a Printer object and demonstrates how the queue works in practice. It first adds two jobs to the queue and processes one of them individually using processJob(). Afterward, it adds two more jobs to the queue and uses processAllJobs() to process everything that remains. Finally, it calls processAllJobs() a second time to show how the program behaves when the queue is already empty. Overall, the program demonstrates the use of a custom-built queue data structure to manage and simulate the handling of print jobs in the order they were received.

Why?:
A linked list works well for the printer job queue because it can grow or shrink dynamically as jobs come in, without needing a fixed size set in advance. It also avoids wasting memory, since nodes are only created when a job is actually added. Adding a job to the back and removing one from the front both happen in constant time, O(1), since it only involves updating pointers rather than shifting elements like an array would require. This front-to-back structure also naturally fits the first-in-first-out order that a printer needs to follow when processing jobs.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar Correction, Summarization**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I have made a draft of the code analysis and the reason of using linked list and let chatgpt correct my grammar and summarize it**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance

with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.